

Today: parallel pre-scan

Prefix sum - sum of all items in an array up to that point

7	3	15	10	13	18	6	4
---	---	----	----	----	----	---	---

{array

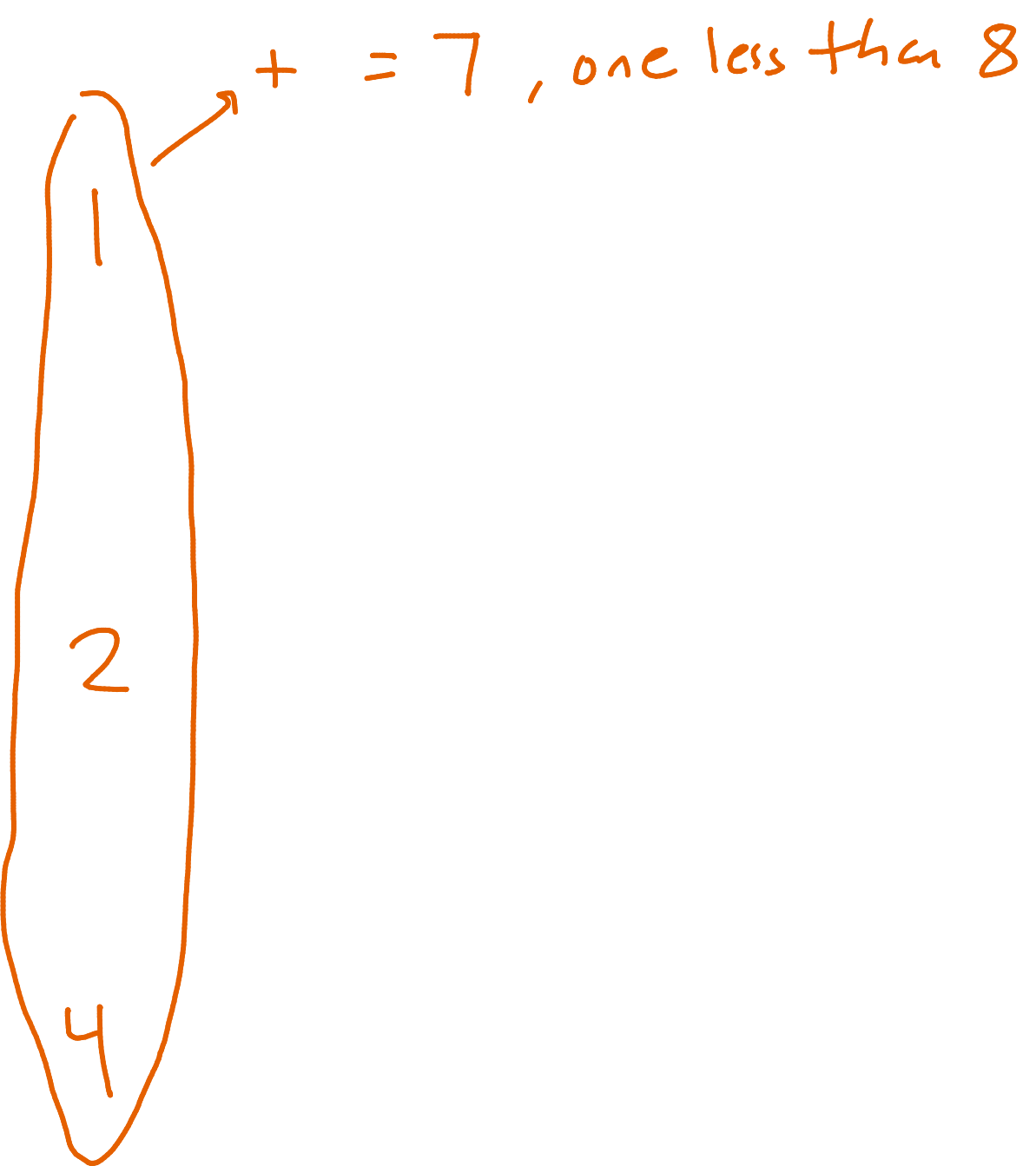
7 10 25 35 48 66 72 76 {prefix sum = "scan"

We're going to instead calculate "pre-scan" - sum of all values up to but not including the current one.

0 7 10 25 35 48 66 72 pre-scan = pre-prefix sum

Sequential is easy, how do you do parallel?

Pass 1: "up" pass - build a binary tree out of the data where every node is the sum of its two children.



(we're assuming n is a power of 2)

↑ build up from bottom

8

← orig data

Analyze pass 1.

Work: - every value in list I have to read, and calculate a sum for each pair $\rightarrow O(n)$ work for bottom layer

(Fact: # of nodes in a complete binary tree above the bottom row = # of nodes in bottom row - 1)

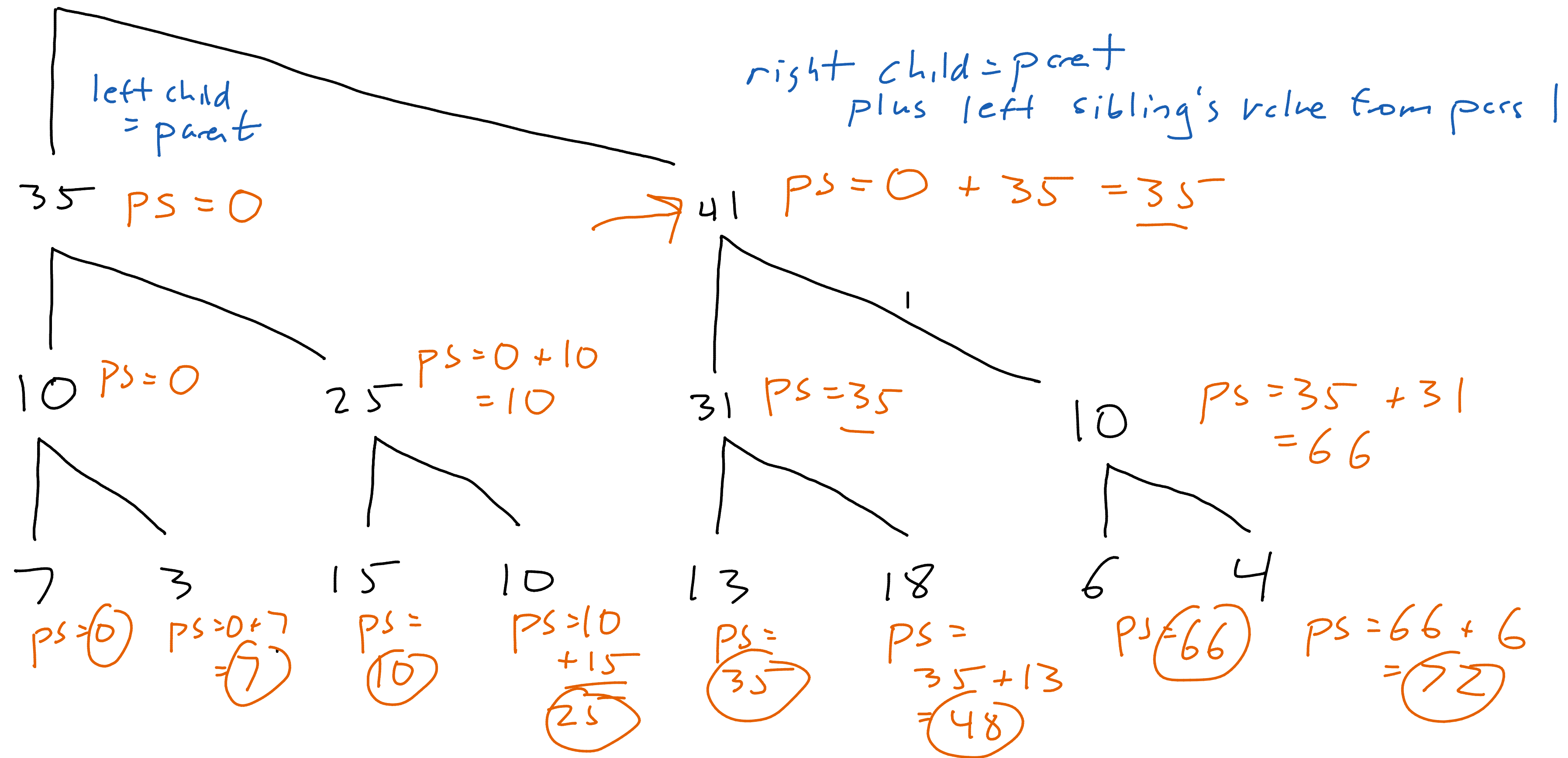
work for the rest of tree is just $O(n)$ again

$\Rightarrow$ so work is $O(n)$ total

Span: the value of each node depends on children
- seq path of length of the depth of the tree $\rightarrow O(\log n)$

parallelism
= $O(n / \log n)$

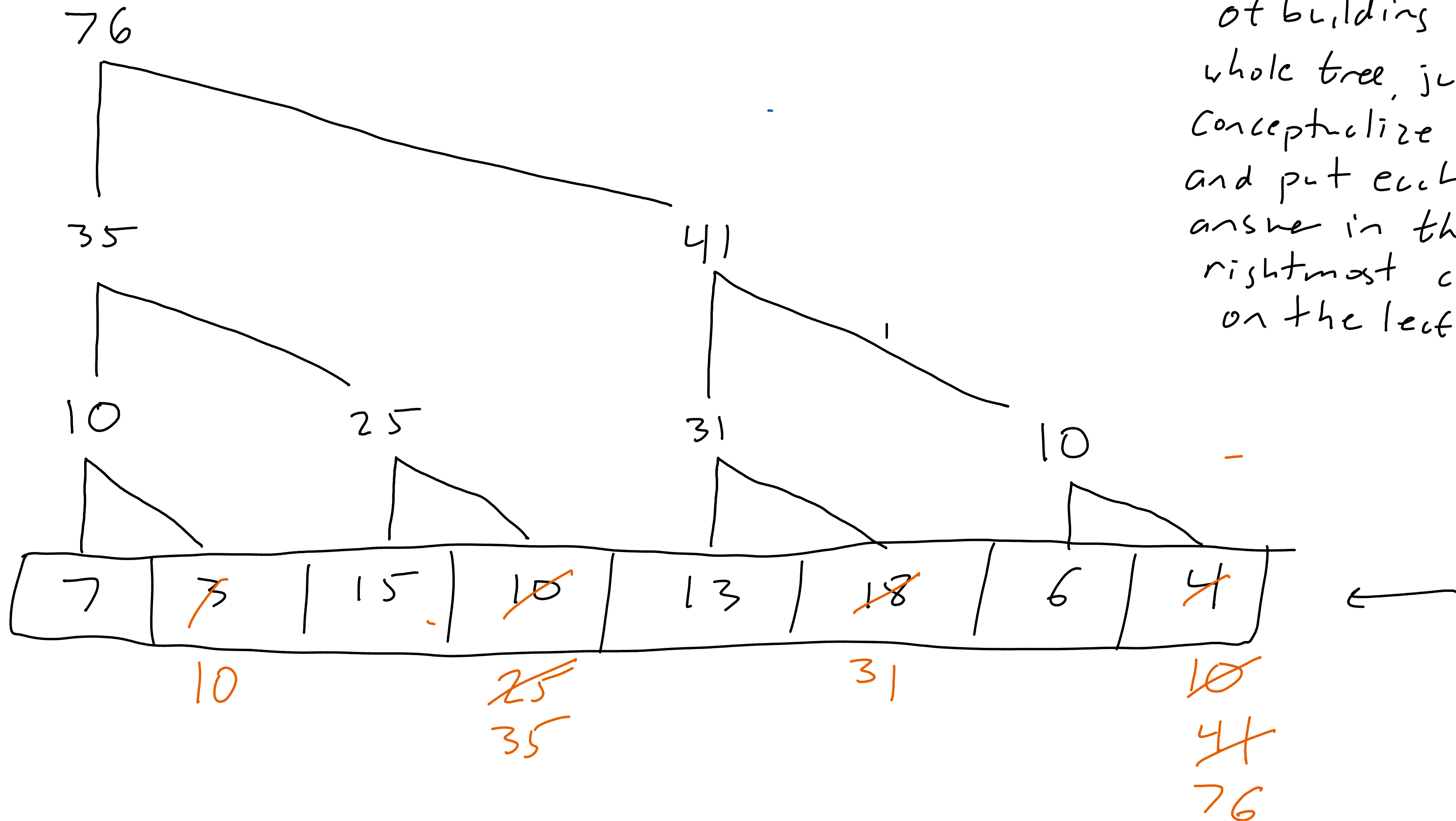
Pass 2: "down pass" Fill in actual pre-scan values from the top.
 76 pre-scan = 0 ← sum of all values in array to the left of here



Work: each node we add some values
 # of nodes is $O(n)$

Span: value for each node depends on parent - sequential chain depth of tree $\rightarrow \log n$
 $O(\log n)$ Parallelism $O(n/\log n)$ exponential speedup

Pass 1: instead
of building
whole tree, just
conceptualize it,
and put each
answer in the
rightmost child
on the leaf level



Pass 2: as you update each value, put in its matching spot

